CHAPTER 12 · PHASE 3: SECURITY & ACCESS CORE

Teleport: Zero Trust Access & Session Recording

Replace direct SSH with an authenticated, authorized, and fully recorded proxy — every session, every command, auditable.

Time estimate: 4–5 hours

Deployed on sec-vault-01

WHAT WE'LL COVER

Chapter Roadmap



Zero Trust Model

Why proxy-based access beats direct SSH



Install & Configure

Binary install, TLS certs, teleport.yaml, systemd



RBAC & OIDC

Roles for admin/devops/developers, Authentik login



Nodes & Clients

Registering agents, connecting with tsh



Database & Kubernetes

Passwordless DB access, logged kubectl commands



Audit & Recording

Session playback, audit logs, the web UI

Trust-Based SSH vs. Zero Trust

Traditional SSH

```
User has private key
→ SSH directly to server → Shell
```

- ✓ Fast to use
- ✗ Hard to audit
- ✗ No session recording
- ✗ Permanent access — hard to revoke
- ✗ Compromised key = full access

Teleport (Zero Trust)

```
Auth verifies → short-lived cert
→ Proxy records session → Agent → Shell
```

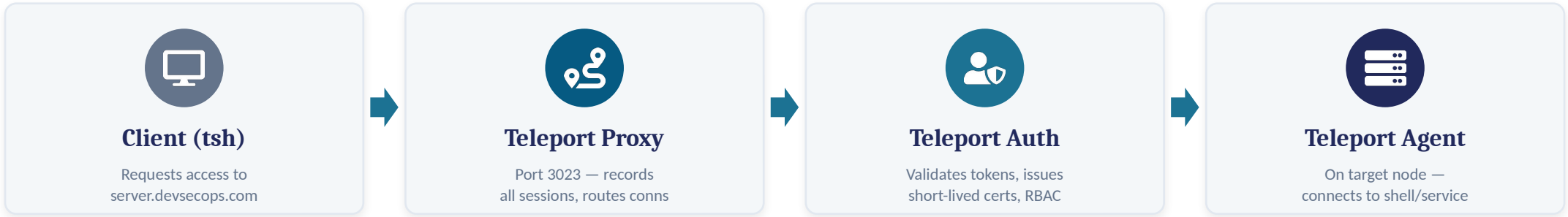
- ✓ Every access authenticated
- ✓ Session recording
- ✓ Fine-grained RBAC
- ✓ Short-lived certs — auto expiration
- ✓ Instant revocation, full audit trail

A Building With a Security Guard

Traditional SSH is a building with permanent key cards. Teleport is a building with a security guard who:



Request Flow — Client to Shell



- Proxy is the single entry point — nothing connects to a node directly.
- Auth issues certificates valid for hours, not forever.
- Agents run on every registered node and enforce the role that was granted.

Installing Teleport on sec-vault-01

1 **Download & install the binary**

```
wget .../teleport-v14.0.0-linux-amd64-bin.tar.gz
tar -xzf teleport-*.tar.gz
sudo mv teleport /usr/local/bin/
```

2 **Create system user & directories**

```
sudo useradd --system --home /var/lib/teleport \
  --shell /usr/sbin/nologin teleport
sudo mkdir -p /var/lib/teleport /var/log/teleport /etc/teleport
```

3 **Generate TLS certificates**

```
sudo openssl req -x509 -newkey rsa:2048 \
  -keyout /etc/teleport/teleport.key \
  -out /etc/teleport/teleport.crt -days 1825 -nodes
```

teleport.yaml — Three Services, One File

auth_service

Issues certs, verifies credentials, enforces RBAC

`listen :3025 · session_recording: all`

proxy_service

Entry point for all clients, web UI, kube proxy

`listen :3023 · web :3080 · kube :3026`

ssh_service

Accepts SSH connections on this node

`listen :3022 · labels: env=lab`

```
oidc_connectors:
  - name: authentik
    issuer_url: http://sec-idm-01:9000/application/o/teleport/
    scopes: [openid, profile, email, offline_access]
    claims_to_roles:
      - claim: groups   value: devops   roles: [devops]
```

Run Teleport as a systemd Service

```
[Unit]
Description=Teleport Zero Trust Access Platform
After=network.target

[Service]
Type=notify
User=teleport
ExecStart=/usr/local/bin/teleport start
--config=/etc/teleport/teleport.yaml
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now teleport
$ sudo systemctl status teleport
→ ● teleport.service ... Active: active (running)
```

```
$ sudo journalctl -xeu teleport -n 20
→ INFO Initialized auth service
→ INFO Initialized proxy service
→ INFO Listening on 0.0.0.0:3025
```

RBAC — Three Roles, Three Blast Radii

Role	Session TTL	Logins	Node Access	Notable Limits
admin	24h	root, ubuntu, teleport	env=lab (all services)	Denied env=production
devops	8h	ubuntu, devops	env=lab, service=*	Denied service=auth nodes
developers	4h	ubuntu	env=lab, service=app only	kubernetes: view-only

```
kind: role
metadata: { name: devops }
spec:
  options: { max_session_ttl: 8h, client_idle_timeout: 15m }
  allow:
    logins: [ubuntu, devops]
    node_labels: { env: [lab], service: ["*"] }
    kubernetes_groups: [system:view]
  deny:
    node_labels: { service: [auth] }
```

OIDC — Authentik Issues, Teleport Trusts

1. **Create the application**
   Applications → Create Application — Name: Teleport, Slug: teleport, Provider: OAuth2/OIDC

2. **Configure the OAuth2 provider**
   Client ID / Secret matching teleport.yaml · Redirect URI: .../v1/webapi/oidc/callback · Scopes: openid, profile, email, offline_access

3. **Map groups to roles**
   LDAP groups (devops, admin, developers) → Authentik groups → claims_to_roles in teleport.yaml

```
redirect_url: https://vault.devsecops.com:3080/v1/webapi/oidc/callback
issuer_url:   http://sec-idm-01:9000/application/o/teleport/
```

Adding an Agent to Every Node

```
# 1. Install the Teleport binary on the target node
$ ssh sec-idm-01 'wget .../teleport-v14.0.0-linux-amd64-bin.tar.gz && \
    tar -xzf teleport-*.tar.gz && sudo mv teleport /usr/local/bin/'

# 2. Generate a one-time join token from the auth server
vm# tctl nodes add --roles=node --ttl=1h > /tmp/teleport_token

# 3. Start the node agent, pointing at the auth server
$ ssh sec-idm-01 'sudo teleport node start \
    --auth-server=vault.devsecops.com:3025 \
    --token=$(cat /tmp/teleport_token) \
    --labels=env=lab,service=idm'
```



A systemd unit (teleport-node.service) keeps the agent running and restarts it on failure — the same pattern used for the auth/proxy service itself.

Connecting With tsh

Login (opens browser to Authentik)

```
$ tsh login --proxy=vault.devsecops.com:3023
```

List available nodes

```
$ tsh ls
```

Connect to a node

```
$ tsh ssh ubuntu@sec-idm-01
```

List & play back recordings

```
$ tsh recordings ls
$ tsh recordings play <session-id>
```

Every tsh ssh session is authenticated via Teleport, video-recorded, has every command logged, and enforces the caller's RBAC role — automatically, with no extra steps.

Database & Kubernetes Access

Database Access

```
database_service:
  enabled: yes
  databases:
    - name: postgres
      protocol: postgres
      uri: postgres.devsecops.com:5432
```

Users connect without ever knowing the password:
tsh db connect postgres --db-user=appuser

Kubernetes Access

```
kubernetes_service:
  enabled: yes
  kube_cluster_name: devsecops
  static_labels:
    env: lab
```

Users log in once, then every kubectl command is recorded and logged:
tsh kube login devsecops
kubectl get pods

Audit Logging & the Web UI

Every audit event captures:

User Who accessed the system (john, alice, bob)

Timestamp When it happened

Event user_login, session_start, session_command, session_end

Resource What was accessed (e.g. node: sec-idm-01)

Result Success or failure

Session ID Link directly to the video recording



Web UI

```
$ firefox https://vault.devsecops.com:3080/
```

- View active sessions
- Play back session recordings
- Manage users and roles
- Configure access policies

Common Problems & Fixes

"Connection refused" on port 3023

netstat -tulnp | grep 3023 — then check systemctl status teleport and journalctl -xeu teleport

OIDC login fails

Verify oidc_connectors in teleport.yaml and confirm the redirect URI matches exactly in Authentik

Node won't register

From the node: nc -zv vault.devsecops.com 3025 — then check tctl tokens ls for a valid join token

Session recording not working

grep session_recording /etc/teleport/teleport.yaml — should show session_recording: all

12.16 VALIDATION SCRIPT

validate-chapter-12.sh — What It Checks

Teleport service active

```
systemctl is-active teleport
```

Listening on 3023 / 3025 / 3080

```
nc -z $TELEPORT_IP <port>
```

Web UI accessible

```
curl -sk https://$TELEPORT_IP:3080/
```

Nodes registered

```
tctl nodes ls
```

OIDC configured

```
grep oidc_connectors teleport.yaml
```

Prints a final PASS/FAIL count; exits non-zero if any hard check fails, so it's safe to gate later chapters on it.

Complete Zero Trust Access Model

1	**User Identity**	Authenticate via Authentik (OIDC) — groups: devops, admin, developers
2	**Teleport Authorization**	Verify OIDC token, check RBAC permissions, issue short-lived certificate
3	**Teleport Proxy**	Record entire session (video), enforce role limits, log every command
4	**Access Grant**	Short-lived (4h–24h), automatic expiration, instant revocation
5	**Audit Trail**	Session recording playback, command logging, compliance reporting

12.18 CHAPTER SUMMARY

What We Built

What We Built	Purpose
Teleport Proxy	Entry point for all access
Teleport Auth	Verify credentials, issue certs
Teleport Agents	On each node, accept connections
OIDC Integration	Authenticate with Authentik
RBAC Roles	Fine-grained access control
Session Recording	Video recordings of all sessions
Audit Logging	Complete audit trail

KEY TAKEAWAY — ACCESS IS NO LONGER TRUST-BASED

Before: direct SSH, permanent access, no recording. **After:** proxied, short-lived, fully recorded and logged.

PHASE 3 · SECURITY & ACCESS CORE

Phase 3: Complete ✅

CHAPTER 10

Vault

Secrets management

CHAPTER 11

Vault PKI

Certificate authority

CHAPTER 12

Teleport

Zero trust access

RESULT — COMPLETE SECURITY & ACCESS CORE

- ✓ All secrets encrypted and rotated
- ✓ All certificates auto-issued and auto-renewed
- ✓ All access recorded and auditable

NEXT STEPS

What's Possible Now — and What's Next

With Phase 3 complete, you can:

- Multi-team isolation — teams access only their systems
- Compliance reporting — SOC2, PCI, HIPAA-ready audit trails
- Incident forensics — watch recordings to investigate breaches
- Instant access revocation via certificate revocation

A

Phase 4: CI/CD Supply Chain

GitLab, SonarQube, Harbor — complete secure software delivery

B

Security Deep-Dives

Production hardening, CIS benchmarks, penetration testing scenarios

C

Phase 5: Orchestration

Kubernetes cluster, GitOps deployment, service mesh (Istio)